

Lab 1: Assembly Lab

Overview

The **fast Fourier transform (FFT)** is an algorithm which has a wide range of applications in different fields such as recording, sampling, additive synthesis, and pitch correction software. The Cooley-Tukey algorithm is the most commonly used FFT algorithm. In this lab, we will implement in the RISC-V assembly language an FFT program that uses the radix-2 Cooley-Tukey FFT algorithm, and then optimize the assembly program for higher performance when running it on a cycle-accurate CPU simulator.

Before starting this lab, you are supposed to be equipped with the following:

- Access to a Linux server (suggested approach, set at Lab 0)
- Basic Linux operation (refer to Linux cheat sheet)
- Basic C programming language
- Basic RISC-V assembly

After this lab, you will understand the following:

- Concepts of FFT, particularly the Cooley-Tukey FFT algorithm
- Concepts of simulation, especially the use of Ripes, a RISC-V simulator
- Concepts of code optimization and simple performance analysis

During this lab, you are suggested to refer to the following materials:

- Linux cheat sheet
<https://cheatography.com/davechild/cheat-sheets/linux-command-line/>
- FFT and radix-2 Cooley-Tukey FFT algorithm
https://en.wikipedia.org/wiki/Fast_Fourier_transform
https://en.wikipedia.org/wiki/Cooley%E2%80%93Tukey_FFT_algorithm
- RISC-V ISA
Quick reference card: `./riscv-card.pdf`
RISC-V reference book: `./RISC-V-Reader-Chinese-v2p1.pdf`
[Specification, Volume 1](#)
- Code optimization
https://en.wikipedia.org/wiki/Loop_nest_optimization
https://www.agner.org/optimize/optimizing_assembly.pdf

Provided Infrastructure

Inside this repository, we provide the following files:

```
+-- lab1-handout.pdf --- This document
+-- run --- Our workspace folder
|   +-- gen_twiddle.cpp --- Utility code to generate Twiddle factors
|   +-- lab1-riscv-starter-code.S --- Starter code for RISC-V assembly
|   +-- lab1-c-starter-code.c --- Starter code for C
|   +-- lab1-force.cpp --- brute force implementation
|   +-- simple_add.c --- Sample C code
|   +-- simple_add.S --- Sample RISC-V assembly code
|   +-- simple_add_data.txt --- Data for sample RISC-V assembly code
```

```
+-- sample --- origin input data, and expected output results
| +-- data.in --- The input data that you work on.
| +-- data.out --- The output data that you will produce.
| +-- data.ans --- The correct output data that you need to compare with.
+-- riscv-card.pdf --- A reference card of RISC-V instruction set
+-- RISC-V-Reader-Chinese-v2p1.pdf --- A reference RISC-V manual in Chinese.
+-- X11 Forwarding Tutorial.pdf --- The tutorial of ssh with X11 forwarding
```

Tasks

1 Setup Environment

1.1 Log in to the server

Enter your shell or WSL shell for Windows users. Log in to the server with:

```
$ ssh -Y <your-username>@101.6.96.180 -p 3050
```

where `<your-username>` is replaced by the username you filled in the spreadsheet.

1.2 Build Ripes

```
$ git clone --recursive --branch 2026-lab1-v1.1 https://yao-archlab-f26:glpat-
DIScFk4C7WKTFCWhEgLhIG86MQp10nJongk.01.0z0b55b80@git.tsinghua.edu.cn/yao-archlab-
f26/Ripes.git
$ cd Ripes
$ patch --directory external/VSRTL -p1 < external/patches/vsrtl-cstdint.patch
$ patch --directory external/VSRTL/external/cereal -p1 <
external/patches/rapidjson.patch
$ cmake -B build -DCMAKE_BUILD_TYPE=Release .
$ cmake --build build -j4
```

On the shared course servers, use at most 4 parallel build jobs (`-j4`). Do not use `-j$(nproc)` or unrestricted `-j`, as excessive parallel compilation can exhaust memory and affect other users.

Note that if you are using your own Linux environment instead of the server, besides a C++ compiler and `cmake`, you also need to install Qt5 and some Qt5 libraries. For Debian, the packages required are `gcc-riscv64-unknown-elf`, `qtbase5-dev`, `libqt5charts5-dev` and `libqt5svg5-dev`. For [Nix](#) users, the flake development shell is ready for you. You can also try running Ripes on Windows natively, but I have no idea if it will work. Pay attention to not introducing platform-related or compiler-related behavior in your code. If you have questions, consult the TAs.

Now you have successfully set up the Lab 1 environment.

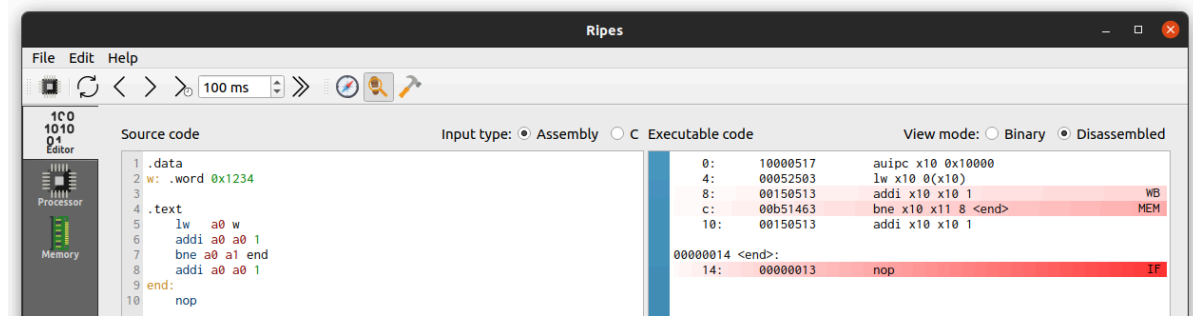
1.3 Ripes Basic Tutorial

Now let's take a look at Ripes, the simulator we are going to use in all three labs. [Ripes](#) is an open-sourced RISC-V simulator, where your RISC-V assembly could get simulated on an x86 machine, instead of being executed by a real RISC-V processor which is uncommon today. For example, Ripes uses a bunch of `int` variables to imitate the functionality of processor registers. And Ripes uses `switch... case...` structures to imitate the instruction decoding. Ripes supports multiple processor models to execute the assembly with different performance. In the next lab, we will learn more about this, and also implement our own processor model!

In our lab, we are going to use a slightly modified version of it that better matches our purpose. So you need to download Ripes from the link above in Chapter [1.2](#), rather than from the public repository.

There are 3 tabs in Ripes:

The Editor Tab



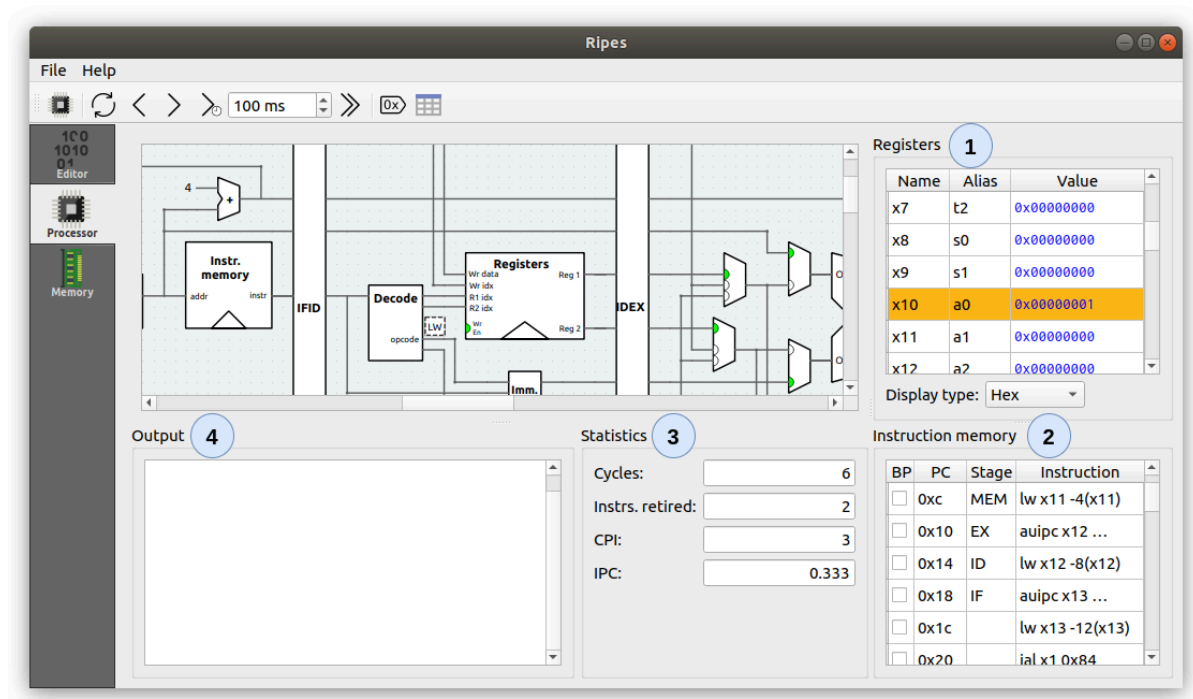
The editor tab shows two code segments. On the left side, you can write an assembly program using the RISC-V RV32 (I/M) instruction sets. Whenever edits are performed in this assembly program - and no syntax errors are found - the assembly code will automatically be assembled and inserted into the simulator. If a C compiler has been registered, the `input type` may be set to `C`. It is then possible to write, compile, and execute C programs within Ripes. We will talk about how to load and execute C programs in Chapter [1.4.1](#).

Next, on the right side, a second code view is displayed. This is a non-interactive view of the current program in its assembled state, denoted as the *program viewer*. We may view the assembled program as either disassembled RISC-V instructions, or as the raw binary code. Clicking on the compass icon at the top toolbar will bring up a list of all symbols in the current program. Through this, it is possible to navigate the program viewer to any of these symbols.

Ripes is bundled with various examples of RISC-V assembly programs, which can be found under the `File` -> `Load Examples` menu.

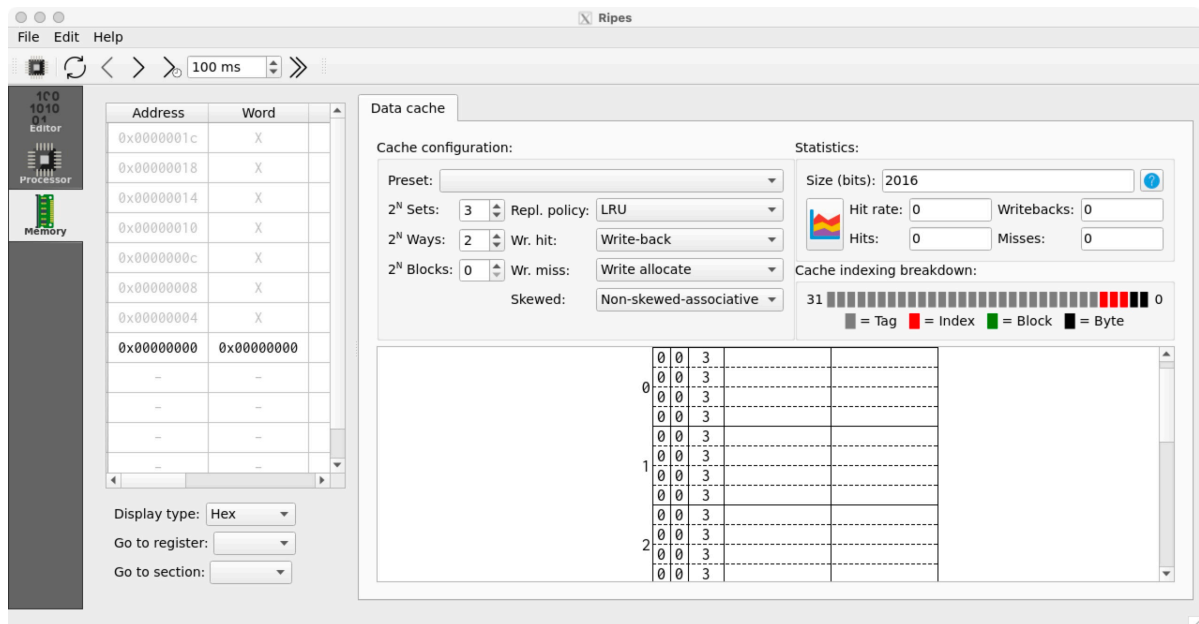
The Processor Tab

The processor tab is where Ripes displays its view of the currently selected processor, as well as any additional information relevant to the execution. You can find the contents of registers (1) and instruction memory (2), as well as the statistics of the execution (3) here. The output console (4) shows the output of the executed program.



The Memory Tab

The memory tab displays the processor's entire addressable address space in the left column and shows the statistics of Ripes' cache simulator in the right column. In the right column, the cache parameters could be configured through the GUI, which will be explored more in Lab 3. However, in Lab 1, to ensure consistency in performance testing, these cache settings must remain unchanged. Please use the default configurations for the data cache: 8 (2^3) sets, 4-way (2^2) associativity, block size 1 (2^0), LRU policy, write-back, write-allocate, and non-skewed associative. The memory tab also provides detailed contents of the runtime cached data including the valid bit and tag bits, and statistics such as the hit rate, which can assist in optimizing your code design.



Controlling the Simulator

The toolbar within Ripes contains all of the relevant actions to control the simulator. The control of the simulator is done by a set of buttons as described below.

Select Processor	Reset	Reverse	Clock	Auto-clock	Run	Display signal values	Show stage table

- **Select Processor:** Opens the processor selection dialog. In Lab 1, you should always choose the 5-stage processor.
- **Reset:** Resets the processor, sets the program counter to the entry point of the current program, and resets the simulator memory.
- **Reverse:** Undoes a clock-cycle.
- **Clock:** Clocks all sequential elements in the circuit for one cycle and updates the states.
- **Auto-clock:** Clocks the circuit with the given frequency specified by the auto-clock interval. Auto-clocking will **stop** once a breakpoint is hit.
- **Run:** Executes the simulator **without** performing GUI updates, to be as fast as possible. Any print `eca11` functions will still be printed to the output console. Running will **stop** once a breakpoint is hit or an exit `eca11` has been performed.

If you want to know more about Ripes, you can find an introduction of Ripes [here](#). But the information above is enough for our labs.

1.4 Load Programs

Ripes can be used to run both C programs and RISC-V assembly programs. First we start Ripes (**when running Ripes, remember to enable X11 forwarding when accessing the server**):

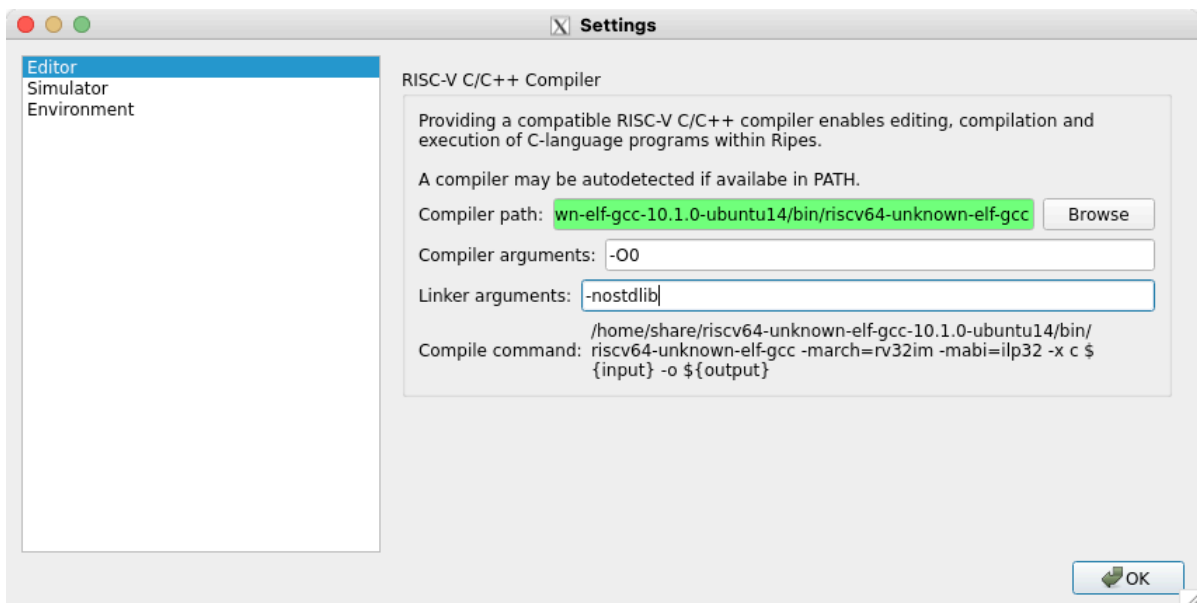
```
$ ./build/Ripes
```

1.4.1 Load a C Program

Now we test a `simple_add` C program. To load programs, you need to **switch to the editor tab** first. Choose `File -> Load Program` on the Ripes window. Choose `Source File` as the file type and open `run/simple_add.c`. **You need to specify your own location of the input file (the location of `simple_add_data.txt`).** The C program will be shown on the left.

Then we need to select the compiler to compile the C program to executable machine code. We provide you a compiler in `/usr/bin/riscv64-unknown-elf-gcc`. Now navigate to `Edit -> Settings -> Editor`. It should look like the picture below. If the compiler path haven't been set, you can set the compiler manually. Click `Browse` and choose `/usr/bin/riscv64-unknown-elf-gcc`.

Notice that the compiler arguments should be `-O0` and the linker arguments should be `-nostdlib`.



Next, click the `Compile C Program` button above, which looks like a hammer icon. You will see the compiled result on the right.

Now switch to the processor tab. Click the `run` button. You will see the output result in the console.

1.4.2 Load a RISC-V Program

Choose `File -> Load Program` on the Ripes window. Choose `Source File` as the file type and open `run/simple_add.s`. **You need to specify your own location of the input file (the location of `simple_add_data.txt`).** The RISC-V program will be shown on the left.

This time you do not need to click the compile button.

Switch to the processor tab and `run`. You can see the output result in the console. **If Ripes crashes**, please go back to the command-line terminal where you execute `./Ripes`. You may see it complains about the error. In most cases it is because of the invalid paths of input files.

2 FFT and Radix-2 Cooley–Tukey FFT Algorithm

2.1 FFT Background

The Fourier analysis converts a signal from its original domain (often time or space) to a representation in the frequency domain and vice versa. The **fast Fourier transform (FFT)** is an algorithm that computes the discrete Fourier transform (DFT) of a sequence, or its inverse (IDFT). FFT has a very wide range of applications in different fields such as recording, sampling, additive synthesis, and pitch correction software. Some of the important applications of FFT include:

- Fast large-integer multiplication algorithms and polynomial multiplication,
- Efficient matrix–vector multiplication for Toeplitz, circulant, and other structured matrices,
- Filtering algorithms (overlap–add and overlap–save methods),
- Fast algorithms for discrete cosine or sine transforms (e.g., fast DCT used for JPEG and MPEG encoding and decoding),
- Fast Chebyshev approximation,
- Solving differential equations,
- Computation of isotopic distributions,
- Modulation and demodulation of complex data symbols using orthogonal frequency division multiplexing (OFDM) for 5G, LTE, Wi-Fi, DSL, and other modern communication systems.

Due to the wide range of applications mentioned above, the optimization of the FFT kernel has always been an important problem.

2.2 DFT Defination

Let $x_0, \dots, x_{N-1}$ be complex numbers. DFT is defined by the formula

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N} \quad k = 0, \dots, N-1$$

where $e^{i2\pi/N}$ is a primitive N th root of unity. Evaluating this definition directly would require $O(N^2)$ operations: there are N outputs X_k and each output requires a sum of N terms. FFT is a method to compute the same results in $O(N \log N)$ operations.

2.3 Radix-2 Cooley–Tukey FFT Algorithm

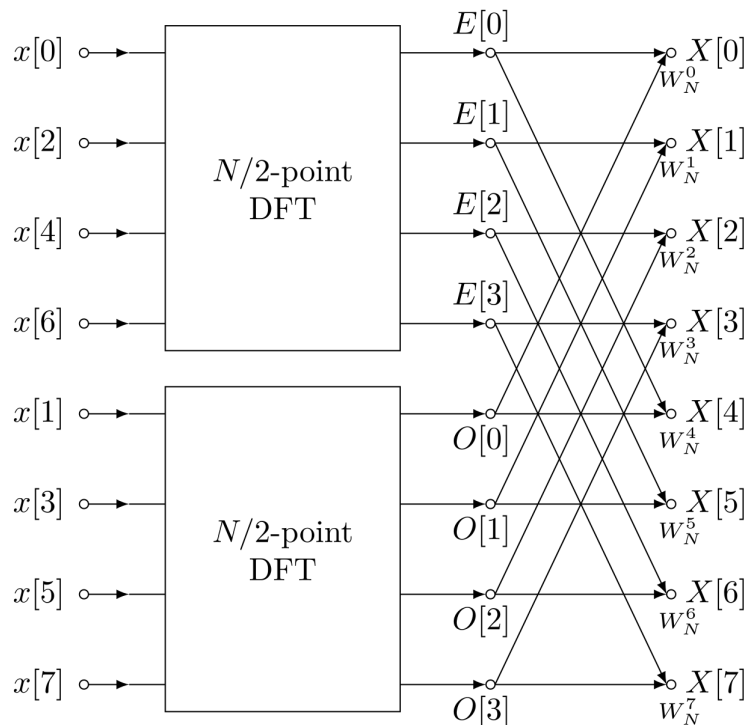
By far the most commonly used FFT is the Cooley–Tukey algorithm. This is a divide-and-conquer algorithm that recursively breaks down a DFT of any composite size $N = N_1 N_2$ into many smaller DFTs of sizes N_1 and N_2 , along with $O(N)$ multiplications by complex roots of unity traditionally called **twiddle factors**.

A radix-2 decimation-in-time (DIT) FFT is the simplest and most common form of the Cooley–Tukey algorithm. Radix-2 DIT first computes the DFTs of the even-indexed inputs $\{x_{2m}\} = (x_0, x_2, \dots, x_{N-2})$ and of the odd-indexed inputs $\{x_{2m+1}\} = (x_1, x_3, \dots, x_{N-1})$, and then combines those two results to produce the DFT of the whole sequence. This idea can then be performed recursively to reduce the overall runtime to $O(N \log N)$. This simplified form assumes that N is a power of two.

Skipping the [derivation and simplification process](#), we denote the DFT of the even-indexed inputs $\{x_{2m}\}$ by E_k and the DFT of the odd-indexed inputs $\{x_{2m+1}\}$ by O_k and rewrite the DFT formula as:

$$\begin{aligned} X_k &= E_k + O_k e^{-\frac{2\pi i}{N} k} \\ X_{k+\frac{N}{2}} &= E_k - O_k e^{-\frac{2\pi i}{N} k} \end{aligned} \quad k = 0, \dots, \frac{N}{2} - 1$$

This result, expressing the DFT of length N recursively in terms of two DFTs of size $N/2$, is the core of the radix-2 DIT FFT. The algorithm gains its speed by re-using the results of intermediate computations to compute multiple DFT outputs. Note that the final outputs are obtained by a $\pm$ combination of E_k and $O_k \exp(-2\pi i k/N)$, which is simply a size-2 DFT.



Note: There are many ways to calculate the above results. You do not need to exactly follow the above description. Feel free to choose the best implementation as long as the results are correct. Actually, this is how you should do during the performance optimization task below.

3 Generate Twiddle Factors

Since the STL libraries are not supported in Ripes (recall that the `-nostdlib` argument needs to be set), STL `sin()`, `cos()`, `Complex`, or `rand()` functions and types can not be used. We thus use the `run/gen_twiddle.cpp` file to generate the twiddle factors in advance. We have generated the twiddle factors for $N = 128$ and have stored them in the global array `expTable` in `lab1-c-starter-code.c`, where `expTable[k]` represents the complex number $e^{-2\pi i k/128}$ according to the above formula for FFT (in the fixed-point representation as discussed below). You can also use these twiddle factors on smaller power-of-two N values without generating new arrays, because `expTable_128[2i]` is equal to `expTable_64[i]`, and so on.

Note: The C starter code provides all 128 twiddle factors, while the assembly starter code initializes only the first 64. The recommended algorithm only uses the first half. If you need the second half, use `expTable_128[64+k] = -expTable_128[k]`, for $0 \leq k < 64$.

4 Use Fixed-Point Format to Represent Fractional Numbers

We can't use floating-point instructions in Ripes since the RISC-V RVF/RVD/RVQ extensions are not supported. We use fixed-point representation with a `scaling factor = 8192` instead. This means that every number in our program will be its actual value multiplied by 8192 and rounded into an integer. The input data, answer data, and twiddle factor data we provide are all in such a format. Due to precision errors, there may be small differences among the answer you output, the answer the brute force program outputs, and the answer we provide. Reducing precision errors is outside the scope of this lab, so any

relative error within 1% or absolute error within 0.005 (i.e., 41 in our fixed-point expression) will be considered the same result and receive full credits.

Hint: You need to divide the integer output by 8192 after each fixed-point multiplication. Think of why.

Note that when running in Ripes, its Write File function will not recognize negative values, but simply output the unsigned values. That means negative values will be written to files in their 2's complement representation. So don't be surprised by large integers in your output file.

5 Write Radix-2 Cooley–Tukey FFT in C Programming Language

5.1 Implementation Description

We have provided you a basic framework to start with: `tab1-c-starter-code.c`. It contains the following global variables/types:

- `N`: The length of the given array. **`N` is no larger than 128 and is a power of 2** in all of our tests.
- `x`: A complex number array that contains the original input data, and also the processed data after performing the function `dft_transform_impl()`. **You do not need to read the input data into `x`.** We have done this in the function `data_input()`.
- `expTable`: A complex number array of length 128 storing the twiddle factors ($e^{-\frac{2\pi i}{128}k}$ for each k). This array is fixed and will not be affected by `x`.
- `read_path`: The path to the input `.in` file. **You need to change it to specify your own location of the input file.**
- `write_path`: The path to the output `.out` file. **You need to change it to specify your own location of the output file.**

We use the type `struct CP` to store complex numbers (e.g., array `x` and array `expTable`). `CP.x` refers to the real part and `CP.y` refers to the imaginary part.

We have implemented the following functions to handle read/write data from/to files:

- `data_input()`: A function to read input data to the `x` array from `sample/data.in`. **You don't need to modify this function.**
- `data_output()`: A function to print the entire processed result `x` to the console and to `sample/data.out` file. **You don't need to modify this function.**

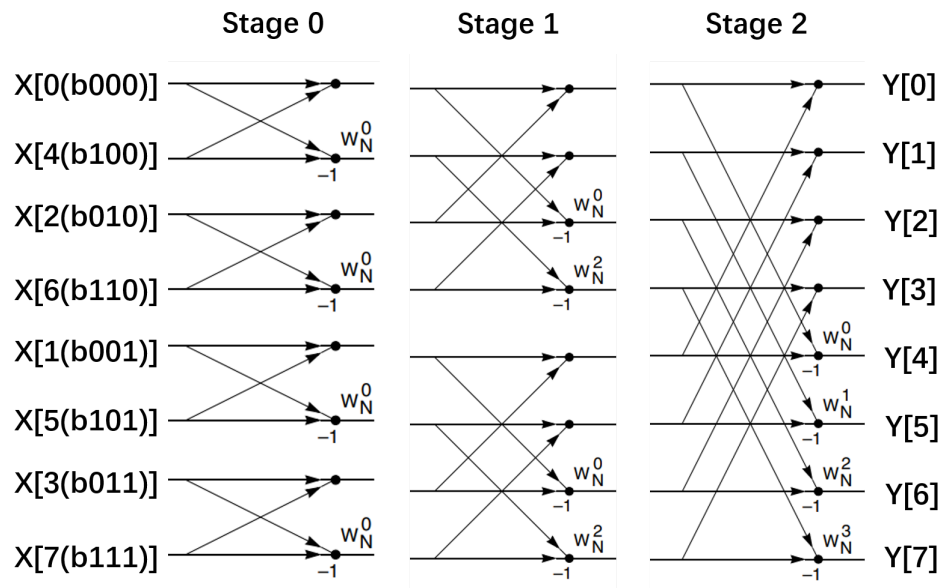
You need to implement one critical function marked as `TODO`:

- `dft_transform_impl()`: Your implementation of the FFT algorithm as described above. The calculation is supposed to be in place, which means the input and output arrays are both the global array `x`.

After finishing your implementation, name the file as `tab1-c.c`.

5.2 A Recommended Implementation Approach

We recommend the following way to implement FFT, as shown in the following figure for $N = 8$. It first shuffles the elements of X into the desired positions, and then sequentially performs the computation of $\log N$ stages.



The following pseudo-code may be helpful:

```
function fft():
    // Get the reverse bit x array in the left column
    for i <- 0 to N-1 do
        // reverse_bit() reverses every bit in the binary representation.
        // You can see the N=8 example in the above figure.
        x[i] <- x[reverse_bit(i)]

    // Iterate the log(N) stages
    for size <- each powers of 2 from 2 to N do
        halfsize <- size/2
        expTable_step <- N/size

        // Iterate each group in this stage
        for i <- 0 to N in stride size do
            twiddle_k <- 0

            // Iterate each top half result of this group
            for j <- i to i+halfsize-1 do
                x[j+halfsize] <- x[j] - (x[j+halfsize] * expTable[twiddle_k])
                x[j] <- x[j] + (x[j+halfsize] * expTable[twiddle_k])
                twiddle_k <- twiddle_k + expTable_step
```

Note that you do not have to follow this approach, as long as your implementation has the desired complexity of $O(N \log N)$.

5.3 Test Correctness

To test the correctness of your C program, enter the directory where your C program resides. Then run

```
$ gcc -DCOMPILER_X86 lab1-c.c -o lab1-c
$ ./lab1-c
```

Your program will print the result to the output file at the path you give. Check the result and see whether the result is correct. We have provided you a correct processed file named `data.ans` in `lab1/sample`.

Hint: Your program can also run in Ripes. If you want to run this C program in Ripes, please refer to Chapter [1.4.1](#). In Ripes, you will be able to see the cycle count and other information directly.

Hint: A good way to test the correctness of your code is to design some small test cases and see whether your code outputs the correct answer.

6 Write Radix-2 Cooley–Tukey FFT in RISC-V Assembly

Sometimes the compiler is not able to generate the most efficient assembly code. That is the time when we prefer to write in assembly languages directly. Let's see which implementation performs better.

Again, we provide you a basic framework to start with. See `tab1-riscv-starter-code.S`. Complete all `TODO`s. Finish your implementation and name the file as `tab1-riscv.S`. The `.S` code reads and writes data in exactly the **same format** as C code. The data is stored in the X array in the same format as a C struct (with real and imaginary parts interleaved). The Y array, located in the data segment, is available for implementations that are not fully in-place. If your code operates fully in-place, you can ignore or remove the Y array.

To test the correctness of your RISC-V code, use the same way in Chapter [1.4.2](#) to load and run it.

Your program will print the result to the output file. Check the result and see whether the result is correct.

Hint: The output results between your C and RISC-V implementations should match. Otherwise there must be bug(s) in either (or both!) of them. That is the reason why you should start with simpler C programming.

7 Optimize RISC-V Code

You may (or may not) see the RISC-V implementation outperform the C implementation. But in any case, let's not just call it a day here. You can do better with RISC-V! Your next task is to optimize your RISC-V implementation in two stages. **Both stages are required.** Complete and test the C and unoptimized RISC-V implementations independently before starting this task.

1. **Independent optimization.** Optimize your RISC-V program on your own, test its correctness, and measure its performance. Save this version as `tab1-riscv-manual.S` before using AI.
2. **AI-assisted optimization.** Starting from your own optimized version, use an LLM or AI agent to explore further optimizations. Save the resulting implementation as `tab1-riscv-opt.S`. You may use a web-based LLM or an agent tool, or even try multiple models/agents and compare their effectiveness. Note that we do **not** solely focus on the final performance, so either a free model or a paid service can be used.

You may explore deeper changes to the computation, data layout, and instruction scheduling. Both versions must satisfy the same lab requirements.

Hint: Some tips for code optimization that you may or may not find useful:

- Use shift instructions (e.g., `srai`, `slli`) instead of multiplication/division. Notice the size of the input array is always a power of two.
- You may notice large overheads from memory accesses. Try to reduce memory accesses by reusing as many registers as possible.
- Another way to optimize memory accesses is through loop optimization techniques, such as loop unrolling and loop blocking. For more information, refer to the wiki page at the top.
- You can implement a non-recursive version to avoid recursive function calls, as the above recommended implementation in Chapter [5.2](#).
- Access arrays (such as `expTable`) in a unit stride (i.e., sequentially), inside of varying strides.

- The code size is limited to **100kB**. Since Ripes does not simulate the memory access cost for fetching instructions, excessive loop unrolling might provide a slight performance improvement but this is impactful in reality. We thus do not recommend such implementations that overfit to Ripes.
- **Numerical optimizations are not recommended**, as this lab focuses on instruction selection, memory access, and program organization. If you change the numerical representation or use approximate arithmetic, explain the effect on accuracy and ensure your program still meets the correctness requirements.

Note: You can only use RV32I, RV32M and some pseudo instructions in this lab. SIMD extensions and floating-point extensions are not allowed or supported. Please refer to the RISC-V reference card.

8 (Optional) A Faster Way to Test Assembly Programs

To run your assembly programs, as we introduced above, you load programs in the editor tab. You can debug using the information provided in Ripes in this way.

However, you may be tired of copying and pasting code to Ripes editor. Ripes also has a CLI interface so you can run tests on your console. For example

```
$ ./build/Ripes --mode cli --proc RV32_5S --isaexts M -t asm --src /home/student/yao-archlab-f26/lab1/run/simple_add.S --cycles --iret --cpi
```

The flags `--cycles`, `--iret`, and `--cpi` report total cycles, retired instructions, and cycles per instruction, respectively. Run `./build/Ripes --help` for more detailed usage.

As you can see, you can get the correctness and the performance (cycle count) in this way, but without any debug information. So it is more useful to test your program when you think the program is almost correct.

Write-up

You should write a design document for Lab 1. There is no specific format required, but you should demonstrate how your program works and how you have been progressively optimizing your program, in a clear way. You should at least include the following parts:

Methodology

How you finished your lab. Explain how your assembly program is organized. Show your debugging process if there is anything interesting.

Summarize the optimizations used in both the manual and AI-assisted stages. For each optimization, describe the code changes, whether and why they reduce instruction count and/or CPI, and the measured change in cycles. Identify the versions being compared, and use intermediate versions or comparison experiments where needed. If several changes were measured together, state that clearly.

For the AI-assisted stage, describe the model(s) and tool(s) used, what context and directions you supplied, and how you guided successive changes using test results. Include representative prompts or interaction excerpts, explain your decisions to accept, modify, or reject suggestions, and describe how you verified correctness. You should **not** submit a complete chat history without extracting the key insights.

Results

Report total cycles, retired instructions, and CPI for: 1) the C implementation compiled with `-O0`; 2) the unoptimized RISC-V implementation; 3) your independently optimized implementation (`lab1-riscv-manual.S`); and 4) the AI-assisted implementation (`lab1-riscv-opt.S`).

Question Answering

1. Compile your C code into RISC-V assembly using GCC in the following way: `riscv64-unknown-elf-gcc lab1-c.c -S -nostdlib -O0 -o lab1-c.S`, and vary the optimization level `-O0` to `-O3`. Check the output assembly file `lab1-c.S`, and try to explain what optimization techniques are used by GCC `O3`; you only need to list two techniques.
2. What are the limitations still in your optimized code? Any ideas on how to overcome them? Describe the ideas; no need to implement.
3. Comparing your experience with manual and AI-assisted optimizations, what do you think can be better done by human and by AI tools respectively? How do you think today's AI on assembly code optimization? Specify one or two things that AI **cannot** do good enough.

Submission

Create a folder with the name of `lab1-<student ID>-<first name>-<last name>`, e.g., `lab1-2026111111-xiaoming-wang`. Put the following files in it.

```
+-- lab1-design-document.pdf --- Your design document
+-- lab1-c.c --- C version of your program
+-- lab1-riscv.S --- unoptimized RISC-V version of your program
+-- lab1-riscv-manual.S --- independently optimized RISC-V version saved before using AI
+-- lab1-riscv-opt.S --- RISC-V version after AI-assisted optimization
```

Compress the folder as a `.zip` package by:

```
$ zip -r lab1-<student ID>-<first name>-<last name>.zip lab1-<student ID>-<first name>-<last name>/
```

Upload the zip file to `learn.tsinghua.edu.cn` (网络学堂) by the submission deadline. You may submit for multiple times, and we will grade based on the latest submission.

Grading Policy

Plagiarism is **strictly** forbidden. Contact the TAs if you are in trouble.

- 20% - Correct C implementation.
- 20% - Correct unoptimized RISC-V implementation.
- 20% - Correct optimized RISC-V implementation.
- 25% - Design document: description of optimization methods used and why they help improve the performance; result reporting.
- 15% - Design document: question answering.

Correctness

We will run your program and compare the outputs with the correct answers. More matches mean a higher correctness score. Notice that **we will use different and multiple tests for grading!** So you should thoroughly test your programs before submitting.

Performance References

Advanced AI agents can produce highly optimized code through long, automated optimization loops. We therefore remove the performance score and focus on learning optimization techniques from AI. The cycle counts below are for reference only. Your own results can be different from these, as long as you have reasonable analysis on why some optimizations work or do not work.

Implementation	Cycles	Speedup over baseline
Unoptimized RISC-V	457,785	1.00x
AI-assisted: register reuse and stage fusion	160,923	2.84x
AI-assisted: further data layout and scheduling improvements	104,671	4.37x
AI-assisted: higher-precision twiddle factors and recurrence	99,478	4.60x